

TADs

Implementacion: Interno (entero 32 bits, entero es una objeto)

Interfaz: Es lo que el programador interactua (+,-, operaciones)

Construirlos:

- Procedimientos constructores
- Procedimientos observadores: predicados y extractores

Receta:

- 1) Constructor por cada variante del tipo de dato
- 2) Haga un predicado por cada variante
- 3) Haga un extractor por cada parte de cada variante en el tipo de dato

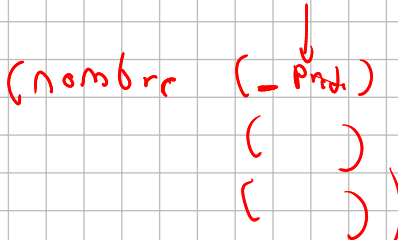
```
#lang eopl
#|
<arb> ::= <int>
      leaf(dato)
      ::= <symbol> <arb> <arb>
      node(key left right)
|#
(define-datatype arb arb?
  (leaf (dato number?))
  (node (key symbol?)
        (left arb?)
        (right arb?)
  ))
```

Constructor

TAD

Predicado

Control



:: Area del programador

```
(define arbol1
  (node
    'a
    (node 'b (leaf 1) (leaf 2))
    (node 'c (leaf 2) (leaf 3))))
```

```
(define suma-arb
  (lambda (exp)
    (cases arb exp
      (leaf (dato) dato)
      (node (key left right)
        (+ (suma-arb left) (suma-arb right))
      )
    )
  )
)
```

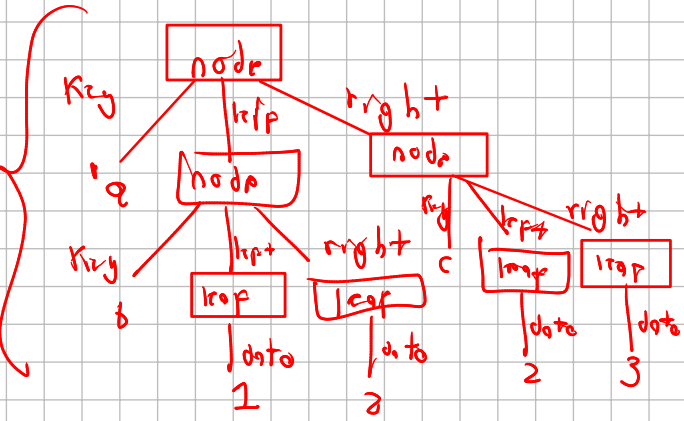
TAD

dato

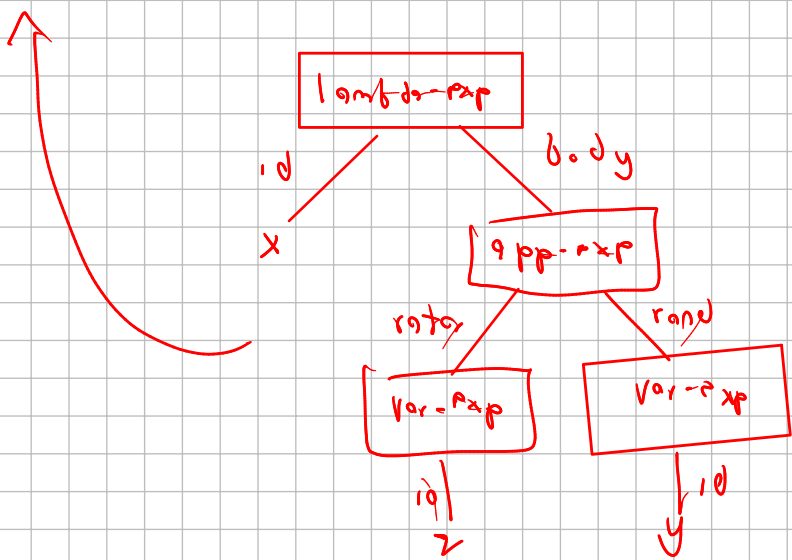
(Cases TAD expresion  
( variante ( - - ) accion )  
                  comp

```
arbol1
#(struct:node
  a
  #(struct:node b #(struct:leaf 1) #(struct:leaf 2))
  #(struct:node c #(struct:leaf 2) #(struct:leaf 3)))
```

AST



```
(lambda-exp 'x (app-exp (var-exp 'z) (var-exp 'y)))
```



```
<lsst-symbol> ::= '()  
               empty-list()  
               ::= (symbol)* <lsst-symbol>  
               non-empty-list(1st rest)
```

```
Esto es una lista de lista de simbolos  
(define-datatype lsst-symbol lsst-symbol?  
  (empty-list)  
  (non-empty-list (lsst (list-of symbol?))  
    (rest lsst-symbol?)  
  )  
)
```

Sintaxis concreta: Es el codigo fuente que el programador escribe, esta es facil de entender por el ser humano

Sintaxis abstracta (AST) representación del codigo usando la gramática, si el AST se puede producir, no hay errores de sintaxis.

Parser: Sintaxis concreta -> Sintaxis abstracta

Unparser: Sintaxis abstracta -> sintaxis concreta

